

Obligatorisk Oppgave 1 – MySmartHome

Fra første idé til fungerende prototype

v2.0

Du har fått plass i et lite utviklingsteam som bygger en prototype for et nytt smarthus. Den første piloten er en vanlig bolig. Det er nettopp poenget: systemet skal løse små, konkrete problemer som mennesker møter hver dag.

Prosjektet starter enkelt. Først skal noen få enheter kunne styres. Deretter kommer nye ønsker fra brukeren: én samlet oversikt, en enkel morgenrutine og til slutt kontroll fra MyPhone. Hver ny del bygger videre på det du allerede har laget.

Dette er et utviklingsprosjekt. For hver del skal du først tenke gjennom designet, deretter implementere og teste, og til slutt reflektere kort over valgene du tok. Det finnes flere mulige gode design. En egen designreferanse blir gjort tilgjengelig senere, slik at du kan sammenligne den med ditt eget forslag.

1. Arbeidsmåte gjennom prosjektet

Bruk samme arbeidsflyt i alle delene:

- Les situasjonen og kravene som et lite systemutviklingsproblem.
- Lag eller oppdater en enkel designskisse. Du kan bruke UML, en klaseskisse eller begge deler.
- Implementer løsningen i Java med meningsfulle engelske navn på klasser, metoder og variabler.
- Kjør et lite testscenario som viser at kravene faktisk er oppfylt.
- Skriv en kort refleksjon over de viktigste designvalgene.

Du skal ikke bygge et komplett kommersielt smarthus. Hold løsningen liten, lesbar og konsistent med det som er gjennomgått i OBJ2000.

2. Oppgave 1 – Den første kvelden i det smarte hjemmet

Situasjonen

Pilotboligen er klar, men foreløpig er den ikke særlig smart. I stuen står en lampe, og i gangen finnes en termostat. Brukeren vil starte med noe helt grunnleggende: enhetene skal kunne identifiseres, slås av og på og vise sin nåværende status.

Enhetene er ikke like. Lampen har en lysstyrke som brukeren kan endre, mens termostaten har en temperaturinnstilling. Når systemet viser informasjon om en enhet, skal det derfor være mulig å se både det som er felles og det som er spesielt for den aktuelle typen.

Det finnes ingen grunn til å opprette en "generell" fysisk enhet i boligen. Alle faktiske enheter tilhører en bestemt type. Teamet forventer også at flere typer kan komme senere.

Krav til løsningen

- Hver smart enhet skal ha et navn og en av/på-status.
- Systemet skal minst støtte en lampe og en termostat.
- En lampe skal ha en justerbar lysstyrke.
- En termostat skal ha en justerbar temperatur.
- Ulike enhetstyper skal kunne presentere relevant statusinformasjon på en konsistent måte.

- Felles informasjon og oppførsel skal ikke dupliseres unødvendig mellom enhetstypene.
- Designet skal kunne utvides med en ny type smart enhet uten at hele modellen må bygges om.

Design

Lag et første design for denne delen av systemet. Vis de viktigste typene, ansvaret deres, viktige data og operasjoner og hvordan typene henger sammen. Oppdater designet dersom du oppdager en bedre løsning under implementasjonen.

Implementer og demonstrer

1. Opprett minst én lampe og én termostat med forskjellige startverdier.
2. Vis status for begge enhetene.
3. Slå dem på, endre lysstyrken på lampen og temperaturen på termostaten, og vis status på nytt.
4. Vis at forskjellige enhetstyper kan brukes gjennom det de har felles, samtidig som den riktige typespesifikke informasjonen fortsatt kommer frem.

Reflekter kort

1. Hva valgte du å plassere i den felles delen av designet, og hvorfor?
2. Hva måtte være spesifikt for lampen og termostaten?
3. Hva måtte du endre dersom boligen senere får en ny type smart enhet?

3. Oppgave 2 – Ett hjem, én oversikt

Situasjonen

Etter den første testen kommer et helt naturlig spørsmål fra brukeren: «Må jeg virkelig holde styr på hver enhet for seg?» I en ekte bolig vil antallet enheter variere. Nye enheter kan bli installert, og brukeren trenger ett sted som gir oversikt over hele hjemmet.

Systemet skal derfor kunne samle de registrerte enhetene og arbeide med dem som en gruppe. Brukeren vil se hva som finnes i hjemmet, slå en valgt enhet av eller på og kunne slå av hele hjemmet når hun går ut døren.

Lampene og termostatene skal fortsatt beholde sitt eget ansvar. Den sentrale delen av systemet skal koordinere dem, ikke kopiere all funksjonalitet inn i én stor klasse.

Krav til løsningen

- Ett hjem skal kunne holde oversikt over et varierende antall smarte enheter.
- Det skal være mulig å registrere nye enheter etter at hjemmet er opprettet.
- Forskjellige typer smarte enheter skal kunne ligge i den samme oversikten.
- Systemet skal kunne vise alle registrerte enheter og deres aktuelle status.
- Brukeren skal kunne slå en valgt enhet av eller på.
- Brukeren skal kunne slå av alle enhetene med én handling.
- Løsningen skal fortsatt fungere dersom en ny enhetstype legges til senere.

Design

Utvid designet fra Oppgave 1 slik at det også viser den delen av systemet som representerer selve hjemmet og forholdet til enhetene. Tenk spesielt på hvilket ansvar som tilhører hjemmet, og hvilket ansvar som fortsatt bør ligge hos den enkelte enheten.

Implementer og demonstrer

1. Opprett ett hjem med minst tre enheter: to lamper og én termostad.
2. Registrer enhetene i hjemmet og vis en samlet oversikt.
3. Slå på flere av enhetene gjennom hjemmets funksjonalitet.
4. Slå av én valgt enhet og kontroller at de andre beholder sin status.
5. Bruk handlingen for å slå av alle enheter og vis sluttstatus.

Reflekter kort

1. Hvilket ansvar har delen som representerer hjemmet, sammenlignet med de enkelte enhetene?
2. Hvorfor er det nyttig at ulike enhetstyper kan behandles samlet?
3. Hvordan unngår designet at funksjonalitet for en bestemt enhetstype sprer seg unødvendig til resten av systemet?

4. Oppgave 3 – Huset begynner å hjelpe til

Situasjonen

Etter noen dager ønsker brukeren litt mer automatisering. Hun vil at enkelte enheter skal kunne få en enkel planlagt tid. For eksempel kan en lampe være planlagt til kl. 07:00 og termostaten til kl. 06:30.

Samtidig installeres en enkel bevegelsessensor i gangen. Sensoren kan rapportere om bevegelse er registrert, men den trenger ikke planlegging. Dette skaper et nytt designproblem: noen typer enheter har en ekstra egenskap eller evne som andre ikke har.

Prototypen skal ikke styre ekte klokkeslett eller fysisk maskinvare. Det er nok at systemet kan lagre en planlagt tid og vise at riktig objekt har mottatt den.

Krav til løsningen

- Minst lampen og termostaten skal kunne få en enkel planlagt tid.
- Systemet skal kunne arbeide med alle enheter som støtter planlegging på en felles måte.
- En bevegelsessensor skal kunne registreres i hjemmet og rapportere om bevegelse er oppdaget.
- Bevegelsessensoren skal inngå i den samme samlede oversikten som de andre smarte enhetene.
- Enheter som ikke støtter planlegging, skal ikke måtte inneholde irrelevant planleggingsfunksjonalitet.
- Alle enheter skal fortsatt kunne vises i hjemmets samlede oversikt.
- Løsningen skal gjøre det mulig å legge til en annen planleggbare enhet senere uten å endre hvordan planlegging brukes i resten av systemet.

Design

Utvid modellen slik at den nye funksjonaliteten får en naturlig plass. Oppdater designskissen og vis tydelig hvilke typer som deler vanlig smart-enhet-oppførsel, og hvilke typer som i tillegg støtter planlegging.

Implementer og demonstrer

1. Legg til minst én bevegelsessensor i hjemmet sammen med en lampe og en termostad.
2. Planlegg lampen og termostaten til to forskjellige tidspunkt.
3. Vis at de planlagte tidene kan observeres i systemet.
4. Vis status fra bevegelsessensoren.

- Behandle enhetene som støtter planlegging samlet i en liten test og utfør den samme planleggingsoperasjonen på dem.

Reflekter kort

- Hvorfor valgte du å modellere planlegging slik du gjorde?
- Hvordan viser løsningen din at en enhet kan være en smart enhet og samtidig ha en ekstra evne?
- Hva ville blitt vanskeligere dersom alle smarte enheter måtte ha planleggingsfunksjonalitet?

5. Oppgave 4 – Smarthuset flytter inn på MyPhone

Situasjonen

Prototypen fungerer fra testprogrammet, men brukeren vil selvfølgelig ikke åpne Java-kode hver gang hun vil slå av lyset. Neste ønske er derfor tydelig: «Kan jeg styre hjemmet fra telefonen?»

MyPhone skal kunne installere og starte apper slik du kjenner fra prosjektet tidligere i kurset. Den nye SmartHome-appen skal være brukerens inngang til det eksisterende hjemmet. Når appen åpnes, skal den vise enhetene og gjøre det mulig å utføre de viktigste handlingene.

Det er viktig at appen arbeider med det samme hjemmet og de samme enhetsobjektene som allerede finnes. En endring som gjøres gjennom appen, skal kunne observeres når hjemmet vises senere.

Krav til løsningen

- Smarthus-funksjonaliteten skal være tilgjengelig gjennom en app som kan installeres og startes på MyPhone.
- Appen skal arbeide med ett eksisterende hjem og dets registrerte enheter.
- Appen skal kunne vise enhetsoversikten og slå en valgt enhet av eller på.
- Appen skal kunne endre lysstyrken på en lampe og temperaturen på en termostat når brukeren velger en passende enhet.
- Appen skal kunne bruke planleggingsfunksjonaliteten på enheter som støtter den.
- Telefonen skal kunne håndtere SmartHome-appen på samme generelle måte som andre apper.
- Løsningen skal være konsollbasert. Det kreves ikke JavaFX eller annen grafisk brukerflate.

Design

Utvid designet slik at telefonen, appen, hjemmet og enhetene inngår i én sammenhengende modell. Vis hvilke objekter som kjenner til hverandre, og hvilke deler som har ansvar for telefon, appstyring, hjemmets oversikt og selve enhetene.

Implementer og demonstrer

- Opprett eller gjenbruk ett MyPhone-objekt og installer SmartHome-appen.
- Start appen fra telefonen og vis at hjemmets enheter blir presentert.
- Bruk appen til å slå på en lampe og endre lysstyrken.
- Bruk appen til å endre temperaturen på termostaten.
- Planlegg minst én passende enhet gjennom appen.
- Vis hjemmet på nytt og kontroller at endringene er gjort på de samme enhetsobjektene.

MyPhone og den grunnleggende app-strukturen fra kurset kan gjenbrukes. Dersom du ikke har et fungerende MyPhone-prosjekt, blir et minimalt startsett med de nødvendige telefon- og appklassene gjort tilgjengelig sammen med oppgaven.

Reflekter kort

1. Hvilket ansvar har appen, og hvilket ansvar ligger fortsatt i hjemmet og enhetene?
2. Hvordan kan telefonen starte SmartHome-appen uten å kjenne alle detaljene i denne appen?
3. Hvorfor er det viktig at appen bruker det eksisterende hjemmet i stedet for å lage kopier av enhetene?

6. Oppgave 5 – Demo-dag: En morgen i MySmartHome**Situasjonen**

Utviklingsteamet skal vise prototypen. I stedet for å demonstrere én metode om gangen vil dere vise en liten historie: hva skjer i hjemmet fra tidlig morgen til brukeren går ut døren?

Denne delen handler ikke om å legge til mange nye funksjoner. Målet er å vise at designet fra de tidligere oppgavene faktisk fungerer som ett system.

Krav til løsningen

- Demonstrasjonen skal bruke minst to lamper, én termostat og én bevegelsessensor.
- Alle enhetene skal tilhøre det samme hjemmet.
- SmartHome-appen skal være installert på MyPhone og bruke dette hjemmet.
- Minst to enheter som støtter planlegging skal ha forskjellige planlagte tider.
- Brukeren skal utføre både en felles handling og minst to typespesifikke handlinger gjennom appen.
- Demonstrasjonen skal avsluttes med en tydelig statusoversikt over hjemmet.

Design

Oppdater det samlede designet slik at det stemmer med den løsningen du faktisk har implementert. Dette er versjonen som skal dokumenteres i den endelige rapporten.

Implementer og demonstrer

1. Sett opp et lite morgen-scenario med dine egne enhetsnavn og startverdier.
2. Installer og start SmartHome-appen på MyPhone.
3. Vis de planlagte tidene og utfør noen naturlige morgenhandlinger, for eksempel slå på lys, endre lysstyrke og justere temperatur.
4. Vis status fra bevegelsessensoren.
5. Avslutt med at brukeren forlater hjemmet og bruker en felles handling for å slå av aktuelle enheter.
6. Skriv ut en siste oversikt som gjør det lett å kontrollere at objektene har riktig sluttstatus.

Reflekter kort

1. Pek på ett sted i løsningen der samme operasjon gir forskjellig resultat avhengig av hvilken type objekt som brukes. Forklar kort hva som skjer.
2. Pek på ett viktig samarbeid mellom to objekter i systemet. Hvorfor trenger disse objektene hverandre?
3. Sammenlign ditt første design fra Oppgave 1 med den endelige modellen. Hva endret du underveis, og hvorfor?

7. Leveranse**8.1 Kildekode**

Lever én ZIP-fil med kildekoden. Ta med alle `.java`-filene som trengs for å kompilere og kjøre prosjektet, inkludert testklassene. Ikke lever `.class`-filer.

Hvis du gjenbraker MyPhone-prosjektet og allerede har samme klasser i de eksisterende pakkene, behold dem. Du trenger ikke lage duplikater.

Kort rapport

Rapporten skal være kort og dokumentere at du forstår løsningen. Bruk den oppgitte rapportmalen ved innlevering. Fyll inn testresultater, skjermbilder, UML-diagram og refleksjoner i de angitte delene av malen.

UML-diagrammet skal vise den endelige hovedstrukturen i løsningen, inkludert viktige klasser og interfaces, sentrale attributter og metoder, samt arv og viktige relasjoner mellom klassene. Du trenger ikke ta med alle konstruktører, getters/setters, testklasser eller små hjelpemetoder.

Bruk hjelpearket «UML og verktøy» dersom du trenger støtte til UML-notasjon, detaljnivå eller bruk av diagrams.net / PlantUML.

Filnavn

Bruk studentnummeret ditt som filnavn: `StudentID.zip` og `StudentID.pdf`.

8. Hva blir vurdert?

Oppgaven vurderer først og fremst om løsningen fungerer og om du viser bevisst bruk av OOP-konseptene fra leksjon 1–4. Det forventes ikke et stort eller perfekt system.

- riktig og tydelig bruk av klasser, objekter og innkapsling
- fornuftige klasseforhold og ansvarsdeling
- riktig bruk av arv og metodeoverstyring
- polymorf bruk av superklasse- og interface-typer
- trygg downcasting når spesialfunksjoner brukes
- at MyPhone, SmartHomeApp og SmartHome samarbeider som beskrevet
- at testprogrammene viser at delene virker
- at UML og refleksjon stemmer med koden du faktisk har levert

9. Innleveringsfrist

Sjekk fristen i Canvas. Hvis du ser at du får problemer med å levere innen fristen, ta kontakt før fristen utløper.